ROB 550 Report Group1

Tzu-Chieh Tai, Venkatakrishnan Amruthur Kesavan, Avantika Rattan
{tzuchieh, akvenky, arattan}@umich.edu

***Abstract*—Robotic manipulation requires robust perception and precise kinematic control. This report presents a pick-and-place system developed in ROB550, integrating RGB–depth data from a LiDAR camera with OpenCV-based detection to operate the Reactor200 5-DoF robotic arm. Optimized depth contour extraction enables real-time object detection, while the Denavit–Hartenberg model with calibrated offsets ensures accurate motion control. Motion planning and normalization strategies enhance precision and extend the workspace, allowing the robot to complete three tasks efficiently and reliably.**

I. Introduction

A robotic arm manipulator is a robotic system designed to replicate the movements and dexterity of a human arm. It consists of linked segments connected by joints driven by actuators, with an end effector that interacts with the environment. Such manipulators are widely used in manufacturing, assembly, medical, and space applications, varying from simple two-joint designs to multi-DOF systems offering high precision and flexibility.

This paper presents work conducted in the ArmLab for ROB550 (Fall 2025) using the RX-200, a 5-DOF manipulator equipped with seven Dynamixel servo motors and custom 3D-printed end effectors. An Intel RealSense LiDAR Camera L515 provided both RGB and depth data for vision-based manipulation.

The project aimed to: 1. Detect and classify small and large blocks by color, size, and shape. 2. Autonomously pick and place these blocks at defined positions and orientations.

The small and large blocks measured 25 mm and 35 mm in height, respectively, with additional distractor objects present in the workspace. Overall, the project highlighted the integration of perception, kinematic modeling, and motion planning for reliable robotic manipulation.

II. Methodology

This section presents the derivation of coordinate transformations, block detection using depth and RGB data, forward and inverse kinematics, as well as the control strategies implemented for the final task.

A. Reconstructing Workspace

To perform the pick-and-place task effectively, coordinate transformations are essential to bridge the gap between the user interface and the robotic arm. Since the exact pose of the arm cannot be directly inferred from the camera view, a sequence of transformations between the world and screen frames is required. The following section describes how these coordinate conversions are achieved.

1) Extrinsic Matrix: The extrinsic transformation is a homogeneous transformation that converts points in the world frame to the camera frame, which will be convenient to convert to the screen frame later. The transformation between world frame and camera frame is as follow:

$$\mathbf{H} = \begin{bmatrix} \mathbf{R} & \mathbf{d} \\ 0 & 1 \end{bmatrix}, \quad \begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix} = \mathbf{H} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}$$

To construct the homogeneous transformation matrix, we measured the camera's rotation and translation relative to the world frame. The relative pose was first estimated manually, while the small tilt angle [Fig. 1] was obtained using the RealSense Viewer tool. Alternatively, the matrix can be computed by solving the PnP (Perspective-n-Points) problem. Using AprilTags as fiducial markers, we apply `cv2.solvePnP()` to estimate precise rotation and translation matrices between the camera and world frames. The results are presented in the following section.

2) Intrinsic Matrix: The intrinsic matrix converts points between camera and screen coordinate and is independent of the environment. The intrinsic matrix K is as follow:

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & u_0 \\ 0 & f_y & v_0 \\ 0 & 0 & 1 \end{bmatrix},$$

where f_x and f_y represent the local lengths of in pixel units along x and y axis, and u_0 and v_0 denote the coordinates of the optical center with respect to image origin, s is the skew parameter. To get the intrinsic matrix, we use the ROS `camera_calibration` package to infer

intrinsic parameters. This method uses a checkerboard as a source of feature points to form a set of equations and estimate the best-fitting intrinsic parameters by solving a least-squares problem. In our experiment, we repeated five times and averaged to get a more stable result.

The transformation between camera frame and image frame is as follow:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z_c} \begin{bmatrix} f_x & s & u_0 \\ 0 & f_y & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{I} \mid \mathbf{0} \end{bmatrix} \begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix}$$

, where depth z_c is measured along the optical axis and expressed in millimeters in our experiment.

3) Pixel to World: Given a world point, we can project it onto the image plane. Conversely, if the depth of an object and its screen coordinates (in pixels) are known, its 3D position in the world frame can be recovered by performing inverse transformations.

The transformation from image coordinates to the camera coordinate system is given by:

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = Z_c \mathbf{K}^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}, \quad \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} = \mathbf{H}^{-1} \begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix}$$

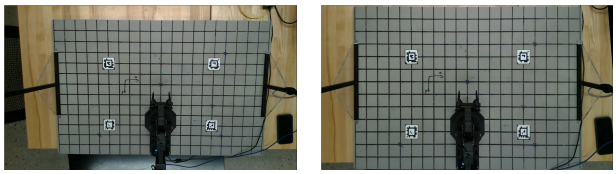
where K is the intrinsic matrix and H represent the homogeneous transformation from world to camera.

4) Homogrphy: The homography transformation converts one 2D image to another while preserving collinearity. As shown in [Fig. 2].

$$\text{Homography Matrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

The upper-left 2×2 submatrix, consisting of $h_{11}, h_{12}, h_{21}, h_{22}$, represents rotation, scale, and shear. The rightmost column, h_{13} and h_{23}, corresponds to translation, while the bottom row, h_{31} and h_{32}, accounts for perspective distortion—that is, how the plane is projected. The bottom-right element, h_{33}, serves as the scale factor.

This matrix is used to reproject the camera view onto the screen, allowing users to observe the entire environment from a unified top-down perspective. In the experiment, we select the centers of four AprilTags as the source points and points near the screen corners as the destination points, while maintaining the aspect ratio of each grid. We then use `cv2.findHomography()` to solve for the homography matrix.



(a) Before homography transformation

(b) After homography transformation

Fig. 1: Comparison of frames before and after homography transformation.

B. Block Detection

Block detection consists of two main stages: segmentation and data extraction. Segmentation involves identifying blocks of interest based on their shapes, after which relevant information — such as RGB values and depth — is extracted from the raw data.

1) Segmentation: To obtain precise block contours, the depth image is used as the primary data source. Each pixel stores its real-time z-coordinate to construct a depth map. To improve efficiency, all pixels are vectorized, and depth values are broadcast across image coordinates, enabling matrix operations without pixel-wise loops. This approach leverages NumPy's optimized linear algebra routines for faster computation. Finally, `cv2.findContours()` is applied to the processed depth map to extract block boundaries, reliably detecting top surfaces while avoiding side interference.

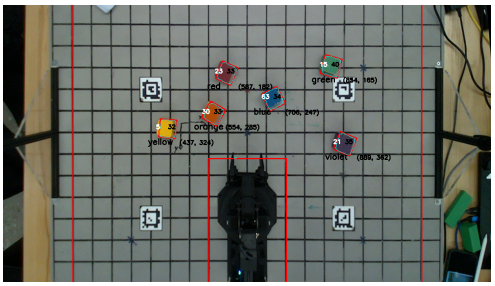


Fig. 2: Detected blocks showing angle (white numbers in degree), center coordinates (in pixels), and height information.

2) Data Extraction: By applying `cv2.minAreaRect()`, we obtain the minimum bounding rectangle for each contour, along with its corresponding area, centroid, and 2D rotation angle. The color of each detected block is determined by sampling the RGB image at the centroid position. To mitigate noise, contours with areas that are either too small or too large are discarded, as they likely correspond to random artifacts caused by environmental variations or human interference. In our experiment, the valid pixel area range for small and large blocks is typically

between 900 and 1700. The color detection compares RGB values with those in a predefined color dictionary by calculating the Euclidean distance, and identifies the color with the smallest distance as the detected color. To mitigate the effects of lighting, a color calibration process is performed prior to task execution.

C. Kinematics and Control

1) Forward Kinematics: The pose of the end effector is essential to manipulate the environment. The pose is the location and orientation of the end effector with the base frame of reference. Forward Kinematics is the analytical (closed form) procedure through which we can find a unique end effector pose given a unique set of joint states. Forward Kinematics is analyzed through the robot body geometry, the length of different linkages, and the joint states. With the ith joint, we associate a joint variable, denoted by q_i. In our case, the robotic arms consist of revolute joints, hence q_i is the angle of rotation of the $(i-1)^{th}$ and i^{th} link.

Since the robotic arm is a kinematic chain of rigid bodies, the final gripper pose is just a series of homogeneous transformations of adjcent links defined as

$$H = T_0^n = A_1(q_1)\cdots A_n(q_n) = \begin{bmatrix} R_n^0 & o_n^0 \\ 0 & 1 \end{bmatrix}$$

where T_0^n is the transformation matrix and $A_i(q_i)$ is the homogeneous transformation from link i-1 to i. The gripper location is calculated as:

$$r_{world} = H * r_{gripper}$$

The gripper pose is calculated by converting the R_n^0 matrix into the Euler angles or the quaternion form.

On this foundation, we use the Denavit-Hartenberg convention to simplify this transformation. In this convention, each homogeneous transformation A_i is represented as a product of four basic transformations

$$A_i = \text{Rot}_{z,\theta_i}\text{Trans}_{z,d_i}\text{Trans}_{x,a_i}\text{Rot}_{x,\alpha_i}$$

where the four quantities θ_i, a_i, d_i, α_i are parameters associated with the link i and the joint i. The four parameters are called link length, link twist, link offset, and joint angle, respectively.

According to D-H convention, we assign coordinate frames based on the following 2 rules (where $o_{i-1}x_{i-1}y_{i-1}z_{i-1}$ is the coordinate frame attached to joint i-1 and $o_ix_iy_iz_i$ is the coordinate frame attached to joint i):

1) x_i is perpendicular to z_{i-1}
2) x_i intersects z_{i-1}

These 2 rules reduce the number of necessary parameters from 6 to 4 for a single transformation.

Refer to Table I and Fig for the D-H parameters and reference frames.

2) Inverse Kinematics: Through Forward Kinematics, we obtained the end effector pose from the joint states. In most robotic applications, the environment is described with respect to the world frame. For robotic arms to perform tasks such as pick and place the location of the objects is provided as input, the configuration is calculated and the task is executed. Hence the robotic arm requires Inverse Kinematics methodology. Inverse Kinematics (IK) is obtaining the joint states $q = [q_1, ..q_n]$ when given the end effector pose $[x, y, z, \psi, \theta, \phi]$.

IK can be solved through 2 methods - Analytical or Numerical. We focus on Analytical methods because of it's simplicity in computation and knowledge of degenerate solutions for a given input.

From Fig 3, ψ, θ_a (angle by which X1 axis is rotated to obtain X2 axis), a (the euclidean distance b/w X1 and X2 axis) and the lengths of the links are known. Using specific triangles and trigonometry, we can find the joint states. The image plane is formed by the 2 axes R, Z. R is the direction faced by the robotic arm due to the rotation of the shoulder joint.

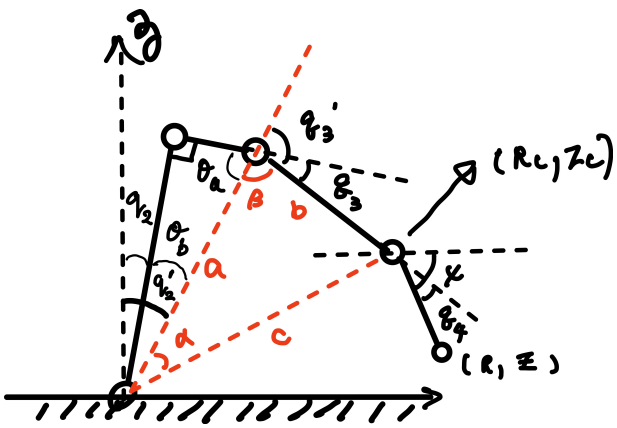


Fig. 3: Inverse Kinematics of q_2, q_3, q_4 in the r-z plane

I will be listing down how all the joint states have been derived. (X, Y, Z, ψ) is the given gripper pose.

$$q_1 = \tan^{-1}(\frac{Y}{X}) - \frac{\pi}{2}$$
$$q_2 = tan^{-1}(\frac{Z_c}{R_c}) - \alpha - \theta_b$$
$$q_3 = {q_3}' - \theta_a$$
$$q_4 = \psi - q_2 - q_3$$

$$
\begin{aligned}
p &= \frac{R_c^2 + Z_c^2 - a^2 - b^2}{2ab} \\
R &= \sqrt{X^2 + Y^2} \\
\theta_a &= 1.326rad \\
\alpha &= tan^{-1}\left(\frac{b * \sin(q_3^{'})}{a + b * \cos(q_3^{'})}\right) \\
q_3{}' &= \pm \cos^{-1}(p) \\
R_c &= R - l_{eff} * \cos(\psi) \\
Z_c &= Z + l_{eff} * \sin(\psi) \\
l_{eff} &= l_4 + l_5 \\
\theta_b &= \frac{\pi}{2} - \theta_a
\end{aligned}
$$

where 'b' is the length of the third link.

Now, I will be explaining how we obtained the different joint states.

- q_1 is +ve when the robot moves counter clockwise as seen in the Fig 4. $\tan^{-1}(\frac{Y}{X})$ is an obtuse angle taken from X axis and since we want the angle formed with +Y axis we subtract by $\frac{\pi}{2}$
- $q_3{}'$ is obtained by considering the triangle containing sides a,b,c and using cosine rule to find beta which is a linear pair to $q_3{}'$. (Just to note it is the entire angle formed by the link with the red dotted line)
- q_3 is found by subtracting $q_3{}'$ by θ_a
- q_2 is found by find α by extending the dotted triangle to include terms like $b * \sin(q_3^{'})$ and $b * \cos(q_3^{'})$.
- q_4 is found when adding up angles along the kinematic chain, i.e. q_2, $\frac{\pi}{2}$ (a part of the 2nd link), q_3, q_4. On the RHS of the equation, we add $\frac{\pi}{2}$ and ψ. The $\frac{\pi}{2}$ in LHS is because we start adding angluar displacement from the vertical axis and hence even on the RHS we must equate to the total angular displacement about the vertical axis.
- q_5 is arbitary and we can take any value. It completely depends upon how we would like to grab the object.

Number of IK solutions vary as:

- Unique solution: When p = 1
- Multiple solutions: When p ∈ [-1, 1)
- No solution: When p ∉ [-1, 1]

The multiple solutions for a given end effector pose can be understood as an elbow down and elbow up configuration of the robotic arm (in a simpler case) as shown in Fig 5.

III. Results

A. Extrinsic Matrix

The hand-measured extrinsic matrix $\mathbf{H}_{hand_measured}$:

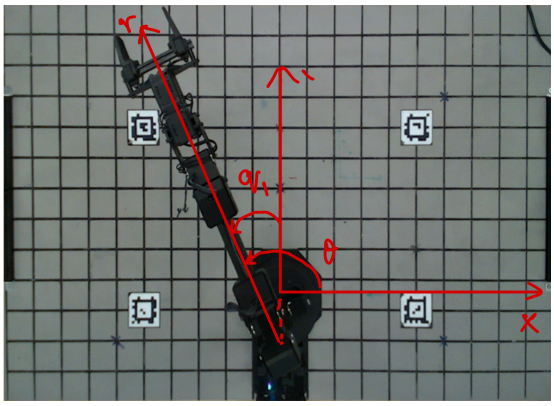


Fig. 4: Approximate Top View of the robot arm for deriving q_1

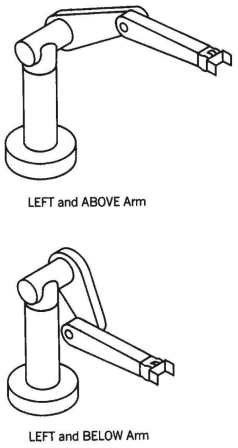


Fig. 5: Elbow up and Elbow down for a PUMA arm which can be understood for our case as well, [1]

$$
\begin{bmatrix}
1.0000 & 0.0000 & 0.0000 & 25.0000 \\
0.0000 & -0.9864 & 0.16425 & 108.6552 \\
0.0000 & -0.1643 & -0.98642 & 1021.7236 \\
0.0000 & 0.0000 & 0.0000 & 1.0000
\end{bmatrix},
$$

The extrinsic matrix was obtained using the PnP algorithm $\mathbf{H}_{pnp}$:

$$
\begin{bmatrix}
0.9998 & 0.01889 & -0.0034 & 29.2411 \\
0.0191 & -0.9945 & 0.1033 & 164.3647 \\
-0.0014 & -0.1034 & -0.9864 & 1021.7236 \\
0.0000 & 0.0000 & 0.0000 & 1.0000
\end{bmatrix},
$$

Difference between the hand-measured and PnP-estimated extrinsic matrices is small. The PnP result shows a close match to the hand-measured parameters, with rotation errors within 2° and a translation deviation

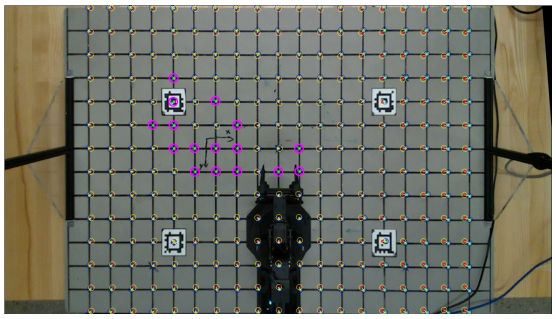


Fig. 6: Verifying calibration result by projecting grid points on image (MSE = 6.3634 pixels)

below 5.5

B. Intrinsic Matrix

The factory intrinsic matrix $\mathbf{K}_{factory}$:

$$\begin{bmatrix} 901.47754 & 0 & 655.13354 \\ 0 & 901.40491 & 355.43893 \\ 0 & 0 & 1 \end{bmatrix},$$

The experimentally calibrated intrinsic matrix $\mathbf{K}_{exp}$:

$$\begin{bmatrix} 926.66504 & 0 & 660.06391 \\ 0 & 923.26092 & 384.54999 \\ 0 & 0 & 1 \end{bmatrix},$$

The manually measured results closely match the factory specifications, with a total deviation of less than 7.55%.

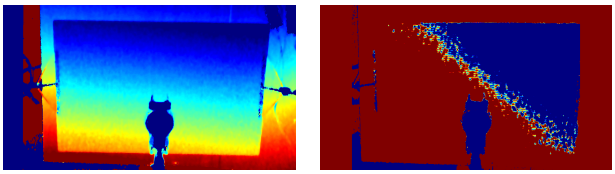
C. Homography Matrix

$$\begin{bmatrix} 1.0530 & -0.0404 & -84.0958 \\ -0.0175 & 1.1382 & -58.2152 \\ 0.00001 & -0.0001 & 1.0000 \end{bmatrix},$$

D. Whole Transformation

The overall transformation includes extrinsic, intrinsic, and homography matrix, and we project grid points to examine the precision of the calibration. The calibration was verified by projecting global points onto the image and checking whether they overlapped with the corresponding world points observed in the image. Grid points were selected for this evaluation. The results are shown in [Fig. 6], where the projected points coincide well with the observed ones after a series of coordinate transformations and homography. By computing the distance between each projected grid point and its corresponding ground-truth position, the mean squared error was found to be 6.3634 pixels, which corresponds to approximately 5 mm in the real world and is considered acceptable.

Before calibration [Fig. 7a], the z values on the board ranged from -30 to 62, due to the tilted angle between the camera and the board, which is not negligible. After calibration [Fig.7b], the deviation dropped to -5 to 14, which corresponds to about 1 centimeter in reality, and is acceptable for our pick-and-place task.



(a) Colorized Heatmap before calibration

(b) Colorized Heatmap after calibration

Fig. 7: Comparison of colorized heatmaps before and after calibration.

E. Block Detection

We use a heatmap, as shown in Fig. 8, to present the accuracy of the detector. Each grid shows how many types of colored blocks (R, O, Y, G, B, V) can be correctly identified. The number of successful identifications ranged from 4 to 6, with an average of 5.27 blocks per grid, corresponding to an overall success rate of 87.9%.

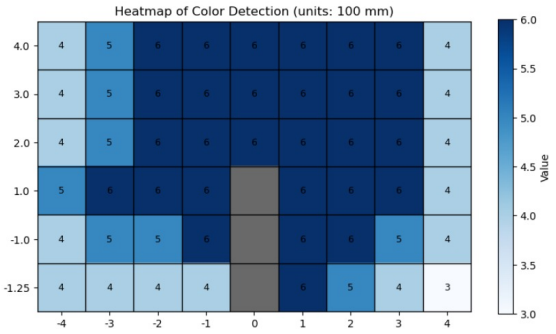


Fig. 8: Heatmap of Color Detection. Units are in 100mm

F. Forward Kinematics

The Table I summarizes the D-H parameters we had obtained. The -ve sign for θ's come from the fact that the +ve angle that the motor operates on is opposite to how we have defined our z-axes.

TABLE I: Denavit–Hartenberg Parameters

Joint i	$\boldsymbol{\theta_i}$ (rad)	$\boldsymbol{d_i}$ (m)	$\boldsymbol{a_i}$ (m)	$\boldsymbol{\alpha_i}$ (rad)
1	$\frac{\pi}{2} + \theta_1$	0.10391	0	$\frac{\pi}{2}$
2	$\theta_a - \theta_2$	0	0.20573	0
3	$-\theta_a - \theta_3$	0	0.2	0
4	$-\frac{\pi}{2} - \theta_4$	0	0	$-\frac{\pi}{2}$
5	$-\frac{\pi}{2} - \theta_5$	0.17415	0	$\frac{\pi}{2}$

Note: $\theta_a = 1.326$ rad is the angle by which X1 axis is rotated to obtain X2 axis as is mentioned in the Fig.9

Fig 9 displays the way we assigned the different frames of reference at each joint.

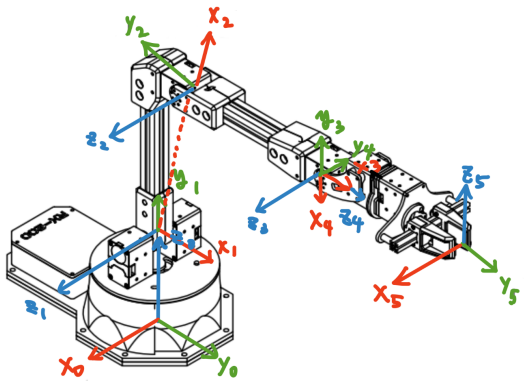


Fig. 9: Frames of reference at each joint

G. Competition

In the final competition, the complete system was evaluated, including coordinate transformations, block detection, and kinematics derivations. Since all components are interdependent, errors may propagate throughout the system. The competition consisted of three events with varying difficulty and scoring criteria.

In Event 1, blocks were sorted and stacked. To ensure consistent grasping, a normalization step was added after each pick-and-place cycle, during which the block was pressed lightly on the ground to secure alignment. Five blocks were successfully stacked in rainbow order within 180 seconds.

In Event 2, both large and small blocks were arranged in rainbow order within 300 seconds. In Event 3, five blocks were stacked within 600 seconds.

IV. Discussion

A. Extrinsic Matrix

The error in the hand-measured extrinsic matrix (see Section III-A) is mainly caused by the high sensitivity of camera orientation and distance measurements. A small angular deviation (e.g., 1°) in the estimated camera tilt can lead to large positional fluctuations of the calibration board. In addition, the relative rotations around the y- and z-axes were neglected in the experiment, which may introduce additional error. To achieve a more precise estimation of the world coordinates, the camera-calibrated extrinsic matrix is considered more reliable

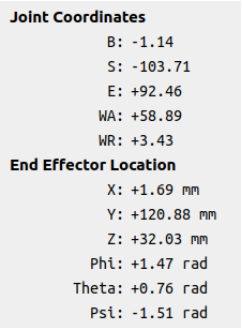


Fig. 10: GUI showing joint coordinates and end effector location at every instant

B. Intrinsic matrix

The calibration results showed (see Section III-B) that the intrinsic matrices obtained experimentally were close to the factory calibration. The source of error in the calibration process may come from noise in corner detection, which can cause reprojection differences of a few pixels and consequently affect the parameters estimated by the least-squares method. The resulting accuracy is acceptable for block manipulation, though it is not as precise as the manufacturer-provided parameters. The factory calibration is preferred for its reliability, because factory uses more advanced and precise equipments to measure it.

C. Forward Kinematics

Implemented our D-H parameters into the kinematics.py file and executed forward kinematics of the robotic arm.

Proceeded to verify if the true end effector location matches with the result obtained from the Forward Kinematics. Verified it by printing the values obtained by FK into our GUI (Fig 10) while switching off the torque in the robot arm and manually moving to the desired locations with the aid of a ruler. We noted down both values and compared it.

We calculated errors based on (all in mm):

$$|x_{FK} - x_{world}|, |y_{FK} - y_{world}|, |z_{FK} - z_{world}| \leq 8$$

It was mentioned in RX200 documentation [2] that the accuracy of the robot should be within 8mm, hence it was justified to use the same limit during our verification.

We attempted recording the verification twice. Initially, we noted huge errors along the y and z direction. After much thought, we hypothesized that the home position as given by the documentation was not practically seen. This is because we observed hang due to weight of the links pulling the arm slighlty down and altering our positions and orientations of our reference frames. We verified this hypothesis by noting the joint states as recorded by the servo motors at the respective joints as seen in the GUI (Fig 10).

We rectified the above issue, by placing each joint at a known joint angle in world frame and compared it with the joint state value from GUI (Fig 10). The difference in θ was noted. Implemented it in our FK algorithm. Now, we verified our tuning by positioning our joint at a known location in world frame and contrasted it against the joint location from the homogeneous transformation until that particular joint. When it was a close match, we proceeded to the next joint. We tuned the θ parameters for joint 1 and joint 3.

TABLE II: Forward Kinematics Verification Results

FK	World Frame	Errors
(0, 175, 100)	(3, 176.73, 99.19)	(3, 1.73, 0.81)
(-300, -75, 25)	(-298.17, -72.72, 25.66)	(1.83, 2.28, 0.66)
(300, -75, 50)	(298.75, -70.49, 52.52)	(1.25, 4.51, 2.52)
(300, 325, 10)	(297.38, 327.06, 13.25)	(2.62, 2.06, 3.25)

Note: All dimensions are in mm. This was our 2nd attempt in recording the FK verification.

D. Inverse Kinematics

To check whether a particular end effector configuration was reachable, we printed the output of the IK algorithm on the terminal.

Number of IK solutions vary as:

- Multiple solutions: When the end effector configuration was reachable and all the joint states were finite values.
- No solution: When the end effector configuration was not reachable, the joint states were assigned nan values.

Our reachability depended upon the ψ value. The reachability range extended along the x,y plane as ψ moved from $\frac{\pi}{2}$ to 0. Similar to rotating our wrists and stretching out our palms to reach faraway objects.

E. Teach-and-Repeat Evaluation

A 10-cycle teach-and-repeat test was conducted to evaluate the repeatability and control precision of the RX-200 robotic arm. As shown in Fig. 12, the end-effector followed nearly identical trajectories across all repetitions. This confirms that the recorded waypoints and interpolation strategy provided stable and consistent replay performance.

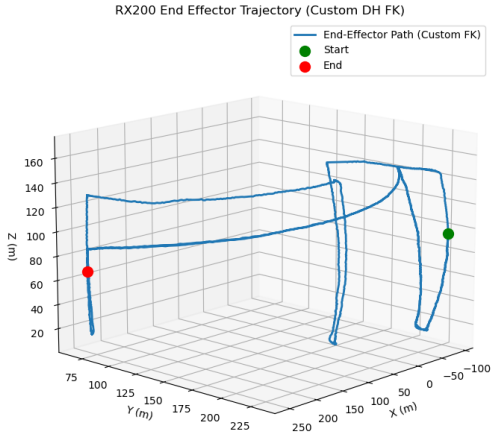


Fig. 11: RX200 end-effector trajectory during 10 teach-and-repeat cycles, showing high spatial repeatability between repetitions.

Minor oscillations near sharp turns were attributed to servo backlash and inertia but did not significantly affect task accuracy. The GUI-based interface—with torque disabled during manual teaching—proved intuitive and responsive, enabling smooth capture and playback of joint states and gripper actions.

Overall, the teach-and-repeat framework achieved high spatial and temporal repeatability, validating the robustness of the waypoint recording and interpolation design. The slight drift observed in later cycles suggests minor mechanical or thermal effects, which could be mitigated through periodic feedback calibration in future work.

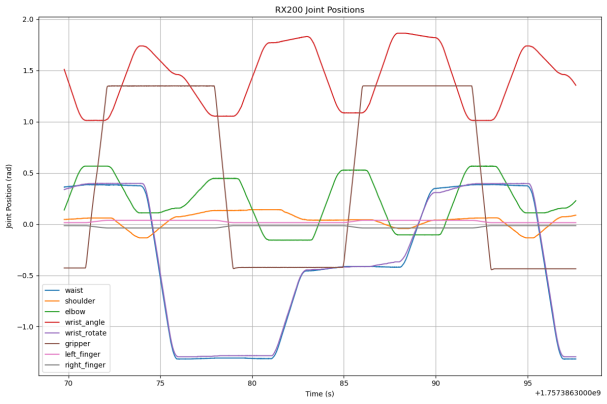


Fig. 12: RX200 joint position trajectories showing periodic motion of all joints during the teach-and-repeat-task.

F. Click to Grab or Place

I will be explaining the algorithm to use mouse clicks on GUI for the robot arm to pick a block and place it to the destination we click at.

- First we assign a variable `self.go_grab`. This variable switches b/w true and false as we click on the GUI.
- Then based on the given mode, we find the world coordinates.
  - First we inverse the homography to obtain the coordinates on the image.
  - Then using depth info and camera intrinsic matrix, we find the points in camera frame
  - Using the extrinsic matrix, we obtain the coordinates in world frame.
- We define 2 set of target points: one at the centre of the block to pick the block and another much above the block to ensure smooth robot motion while picking/placing the blocks.
- We made 3 movements for grabbing: 1) Going above the block, 2) Grabbing the block and 3) Moving with block above.
- 3 movements for placing: 1) Going above desired location, 2) Placing the block and 3) Moving above the block
- For each movement, we found the IK at the desired locations. Then using the obtained joint states we used FK to drive the gripper to desired location.
- Based on the movement, we ensure the gripper was either in grasp or release position.
- In the case of $\psi = \frac{\pi}{2}$, we had to ensure that q_4 turned q_1 rad. Blocks were placed such that its edges were parallel to the world X, Y axes. This ensured the gripper picked the block perfectly.
- Added a clearance term. This enforced that the robot gripper does not hit the base plate while placing the block.
- In the IK, we modified ψ b/w $\frac{\pi}{2}$ at close points and 0 at faraway points. Initially the arm was not able to reach faraway points, now when we modified ψ, reachability is achieved.
- To ensure safe yet swift motion, we set `move_time = 2.0` and `accel_time = 0.6`.

G. Calibration

As shown in [Fig. 6] and [Fig. 8, our calibration results are generally acceptable. However, slight deviations along the z-axis persist, occasionally causing the robotic arm to collide with the ground during grasping or placement. Because the large and small blocks measure $40\,\text{mm}$ and $25\,\text{mm}$ per side respectively, even small perception errors can propagate through forward kinematics and become amplified. From [Fig. 8], a residual tilt can be observed, indicating incomplete correction after extrinsic calibration. In our implementation, AprilTag centers were detected and passed to `cv2.solvePnP()` to compute the extrinsic matrix. Detection errors in AprilTags may introduce significant estimation errors, particularly if a tag is misdetected. To reduce this effect, additional AprilTags can be distributed evenly across the board in future work, allowing the algorithm to solve a least-squares homography that minimizes the influence of any single tag on the final result.

H. Color Detection

We use the RGB values to determine the color of each block; however, this method is highly sensitive to environmental lighting conditions, as RGB represents the absolute intensity of each color channel. To address this issue, we calibrate the predefined color map before each task to ensure that every color is correctly identified. In future work, we plan to adopt the HSV color space, which reduces the effect of lighting variations by representing colors in terms of relative intensity. This approach would eliminate the need for color calibration before every task.

V. Conclusion

Thus, we have performed camera calibration to ensure the world is caputred accurately by the camera and can be observed in the GUI. We have implemented block detection algorithm to accurately detect blocks positions and colours. We have also written the FK, IK and movement algorithms for the robot arm to perform simple tasks such as Click to Grab or Place.

References

[1] M. W. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*. Wiley, 2006.

[2] Trossen Robotics, "Interbotix rx200 robot arm specifications," https://docs.trossenrobotics.com/interbotix_xsarms_docs/specifications/rx200.html, 2025, accessed: 2025-10-16.

[3] University of Michigan, "Rob 550 armlab checkpoint 3 documentation," https://rob550-docs.github.io/docs/armlab/checkpoints/checkpoint3.html, 2025, accessed: 2025-10-16.

[4] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017.

[5] R. M. Murray, Z. Li, and S. S. Sastry, *A Mathematical Introduction to Robotic Manipulation*. CRC Press, 1994.